

慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/05/14)

06 - 系列にラベリングする

情報工学科 准教授 高道 慎之介



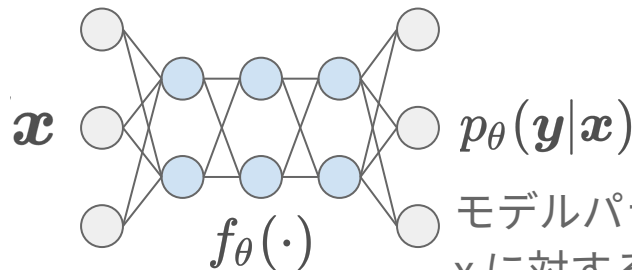
Takamichi Laboratory
慶應義塾大学 高道研究室

講義予定

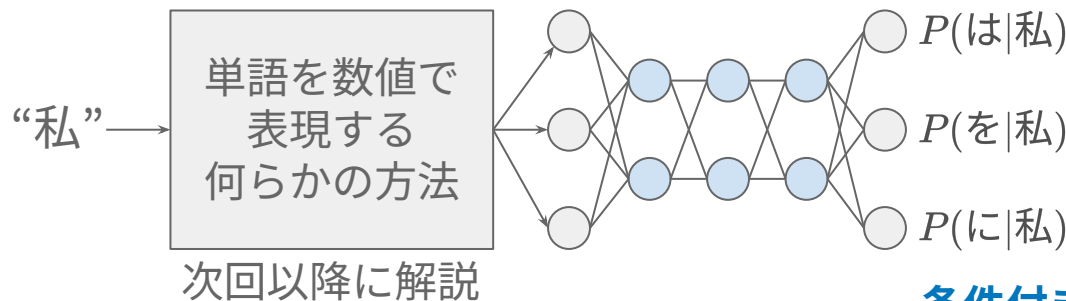
第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	2026/04/16	自然言語処理のための機械学習に入門する	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	2026/04/30	単語をベクトルで表現する	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習 1	これまでの復習
第08回	2026/05/28	Transformer を理解する	大規模言語モデル時代の技術
第09回	2026/06/11	言語モデルを事前学習する	
第10回	2026/06/18	言語モデルを転移学習する	
第11回	2026/06/25	言語を生成 (デコーディング) する	
第12回	2026/07/02	言語やラベルを評価する	
第13回	2026/07/09	自然言語処理から音声言語処理へ	おまけ
第14回	2026/07/16	演習 2	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：ニューラル言語モデル

ニューラルネットワークを 離散確率変数の確率分布を出力するモデルだと考える



モデルパラメータ θ で計算される,
 x に対する y の条件付き確率



単語列で考えてみよう

私 / は / リンゴ / を / 食べる

彼 / は / バナナ / が / 好き / だ

私 / は / 公園 / に / 行く

リンゴ / が / 落ちる

私 / は / 赤い / リンゴ / を / 買う

空 / は / 青い

私 / は / 彼 / と / リンゴ / を / 分けた

彼女 / は / メロン / を / 食べる

私 / は / リンゴ / が / 大好き / だ

犬 / が / 走って / いる

50 単語の文章 (“/” は単語境界)

“私 は” の連続 2 単語の出現確率は？

$$P(\text{私}, \text{は}) = P(\text{は}|\text{私})P(\text{私})$$

“私” の後に “は” が “私” の出現確率
出現する確率

$$P(\text{私}) = \frac{5}{50} \quad (\text{50 単語のうち 5 回出現})$$

$$P(\text{は}|\text{私}) = \frac{5}{5} \quad (\text{“私” に後続する 5 単語のうち 5 回出現})$$

連続 3 単語以上にも拡張可能

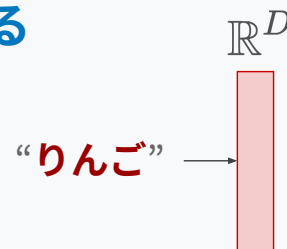
$$P(\text{私}, \text{は}, \text{彼}) = P(\text{彼}|\text{は}, \text{私})P(\text{は}|\text{私})P(\text{私})$$

条件付き確率を計算する,
パラメータ θ を持つニューラルネット

代わりに、密な実数値ベクトルで表すことを考える → 単語埋め込み (word embedding)

あらかじめ定めた次元数 D の密ベクトル (=埋め込み) に写像する

- 単語の集合 (語彙): $V = \{w_1, \dots, w_i, \dots, w_{|V|}\}$
- 写像: $f: V \rightarrow \mathbb{R}^D$



単語埋め込み行列による写像 (lookup)

実質的に行列の特定行の抽出. いわゆる Word Embedding 層.

$$\begin{bmatrix} -0.1 \\ 3.0 \end{bmatrix}_{\mathbb{R}^D} = \begin{bmatrix} -0.1 & 0.2 & 0.3 \\ 3.0 & 2.0 & 1.0 \end{bmatrix}_{\mathbb{R}^{D \times |V|}} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}_{\text{one-hot}} \quad \text{"りんご"}$$

* one-hot ベクトルの次元数爆発は、この時点では解決できていない

言語モデル：単語列から次の単語を予測する統計モデル

$$P(w_{N+1} | w_1, w_2, \dots, w_N)$$

次の単語を出力する確率 これまでの文脈 N 単語が与えられている

慶應義塾大学 理工学部 情報工学科 (Department of Information and Computer Science) は、最先端のコンピュータ技術から、それらを社会に役立てるための知能システムまでを幅広くカバーする **???**

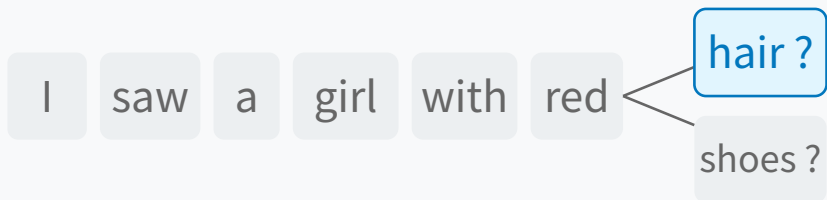
単語予測を繰り返せば文章を生成できる。

- (LLMでは単語ではなくサブワードを扱うが簡単のため単語と表記．扱いは同様)

* 簡単のため「次の」と表記しているが、「前の」や「間の」単語の確率を予測するものも、同様に言語モデルと呼ぶ．次の単語を予測する言語モデルは、自己回帰型言語モデルと呼ばれる。

どんなことに使える？

文章を生成する



テキスト→テキスト変換



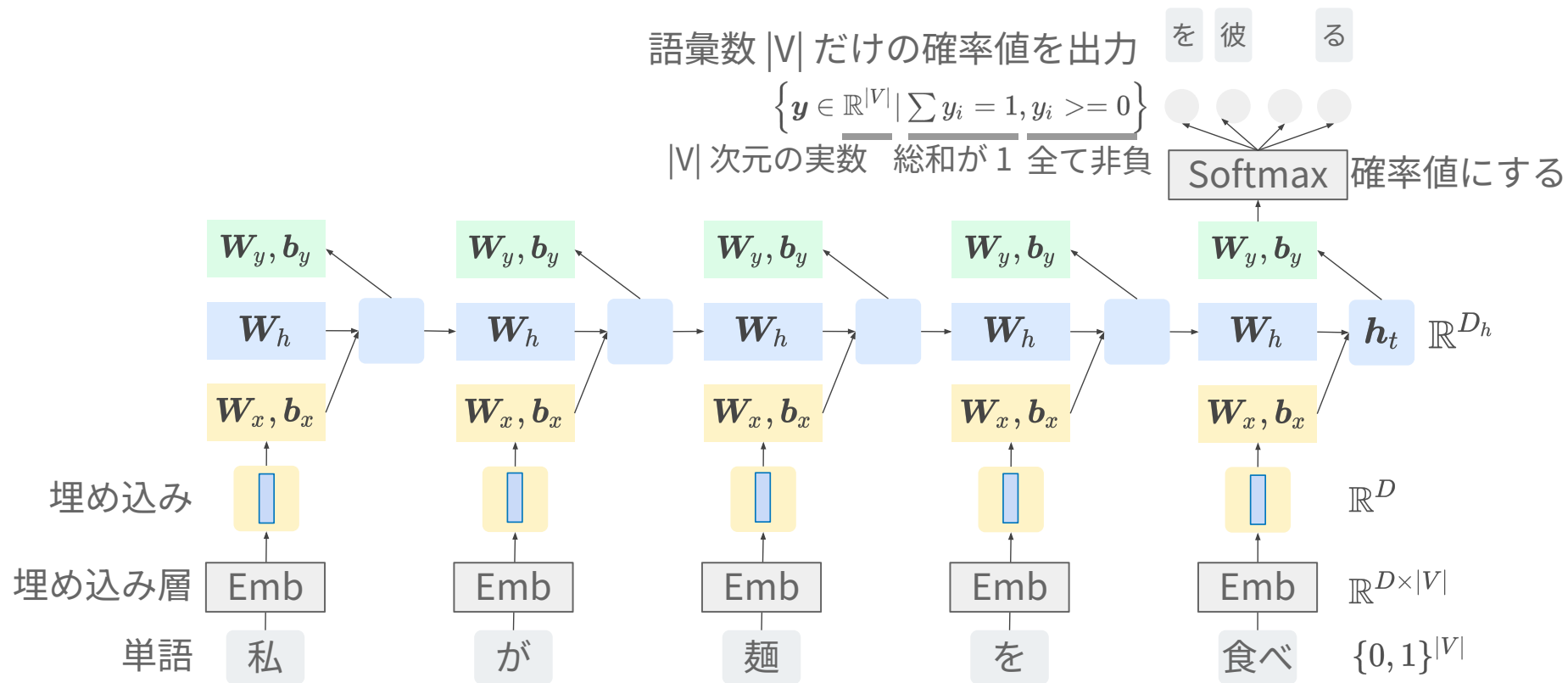
「言語らしさ」を測る



X → テキスト変換



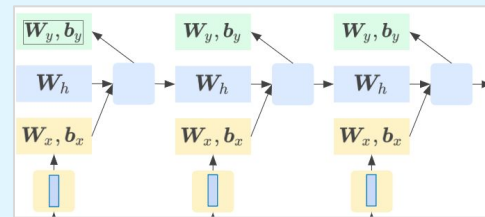
RNN 言語モデル (RNNLM)



RNN LM は n-gram LMの弱点を本当に克服した？

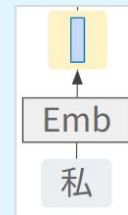
長い文脈 (“理論上は” 無限) を捉えられるようになった！

- 学習時：勾配が過去の全ての単語に勾配を伝える
- 推論時：過去の全ての単語から、ベクトル h を計算



未知の単語列にも対応するようになった！

- 単語埋め込みにより、意味的に類似した単語同士を、近いベクトルとして表現できるようになったため。



Negative log-likelihood (NLL, 負の対数尤度)

文 x の単語数 *サブワードを単位とするならサブワード数

$$\text{NLL} = - \sum_{x \in \mathcal{X}_{\text{test}}} \log P(x) = - \sum_{x \in \mathcal{X}_{\text{test}}} \sum_{t=1}^{|x|} \log P(w_t | w_{<t})$$

評価セットの文 x で総和 文 x の尤度 過去単語から次の正解単語を出力する確率

- **尤度が大きい (NLLが小さい) = その文を高い確率で出力できる**

- 例えば, 当該単語のみ確率が 1 なら, $-\log P(w) = 0$ で最小
- モデル学習時に最小化される目的関数 (クロスエントロピー) と同じ

*後の講義で触れる強化学習の目的関数とは異なる

- 余談

- なぜ「負」? → DNNの目的関数のように「小さければよい」にするため
- なぜ「対数」? → 計算機のアンダーフローを避けるためなど

今日の目的：

系列の要素にラベリングする，系列を別系列に変換する

自然言語処理における系列ラベリング

Sequence labeling in natural language processing

系列ラベリング

単語列のような系列データにラベルをつけること。入力と出力の系列長は同じ。
系列ラベリングのモデルには、どんな (ニューラルネットワーク) モデルを使う？

品詞タグ付け

文中の各単語に品詞
情報を付与

名詞 助詞 名詞 助詞 形容
詞



系列ラベリングモデル



青森 の 林檎 は 甘い

固有表現抽出

文中の人名・地名など
の特定固有名詞を抽出

B(beginning), I(Intermediate), O(thers)

B O O O O



系列ラベリングモデル



青森 の 林檎 は 甘い

チャンキング

単語を意味のある句に
まとめる

{B, I}{名詞句, 形容詞句, ...}

B名 I名 B名 I名 B形



系列ラベリングモデル



青森 の 林檎 は 甘い

単語区切り

単語の境界を推定

B I B B I B B I



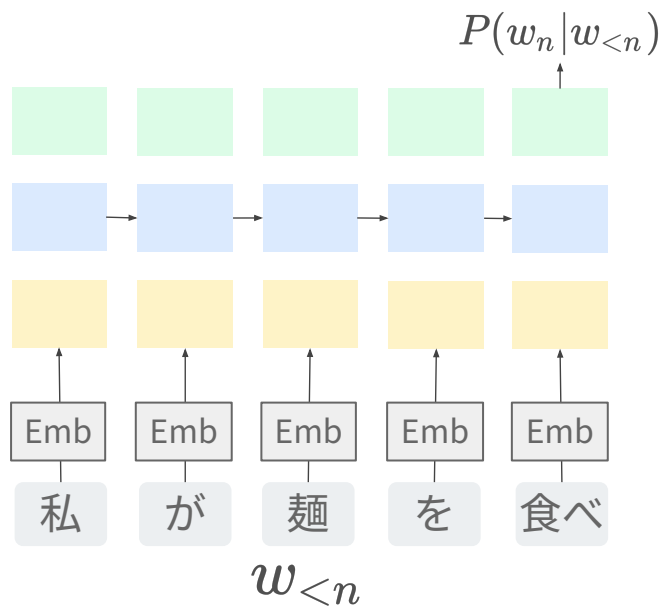
系列ラベリングモデル



青 森 の 林 檎 は 甘 い

その前に言語モデル再考

RNNLM



どの入力
どの出力に
影響するかを
行列で表現

出力側 (目的変数)

入力側 (説明変数)

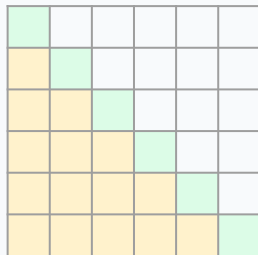


文 $x = [w_1, \dots, w_{|x|}]$ に対する言語モデル尤度

自己回帰型 (e.g., RNNLM)

過去の全単語から計算

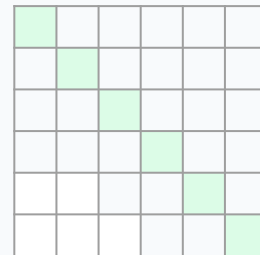
$$P(x) = \prod_{n=1}^{|x|} P(w_n | w_{<n})$$



独立予測 (e.g., 1-gram)

各単語が独立だとみなす

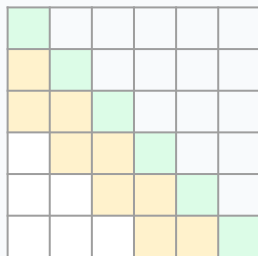
$$P(x) = \prod_{n=1}^{|x|} P(w_n)$$



マルコフ連鎖 (e.g., n-gram, $n > 1$)

過去の N 単語から計算

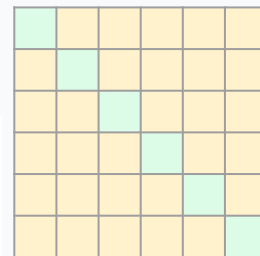
$$P(x) = \prod_{n=1}^{|x|} P(w_n | w_{n-N < i < n})$$



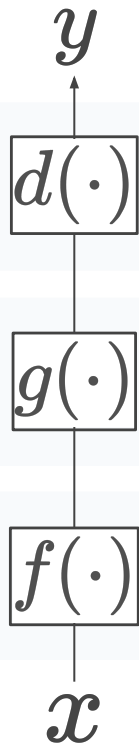
双方向予測 (e.g., BERT)

当該単語以外の全単語から計算

$$P(x) = \prod_{n=1}^{|x|} P(w_n | \dots, w_{n-1}, \cancel{w_n}, w_{n+1}, \dots)$$



復習：x から y の予測



決定 (decision) → 本講義では確率値の最も高い候補を選択

- 出力候補のうちから 1 つを決定すること

スコア計算 (score calculation) → 本講義では出力候補の確率を計算

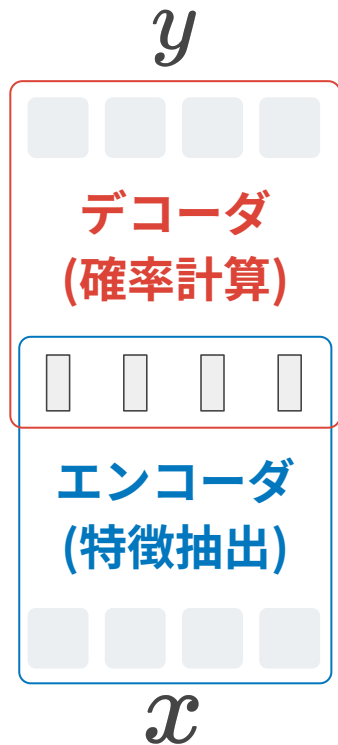
- 特徴に基づいて、出力候補とそのスコアを計算すること

特徴抽出 (feature extraction)

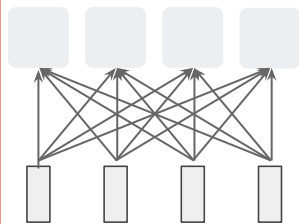
- 意味のある特徴を入力テキストから抽出すること

入力と出力が系列になった場合を考えよう

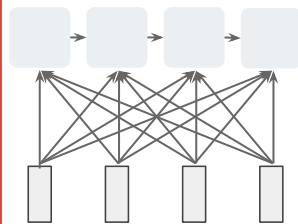
各出力をどの (別) 出力から計算する？



各出力独立

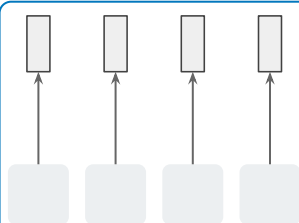


過去出力

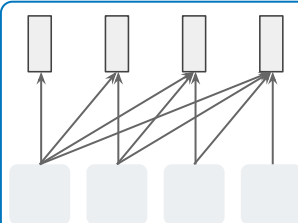


全特徴を使うと
仮定して描写

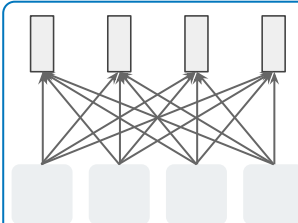
各単語独立



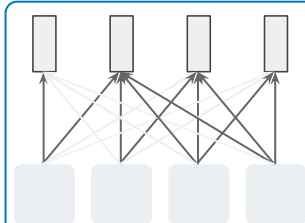
過去単語



全単語



全単語(重み付)

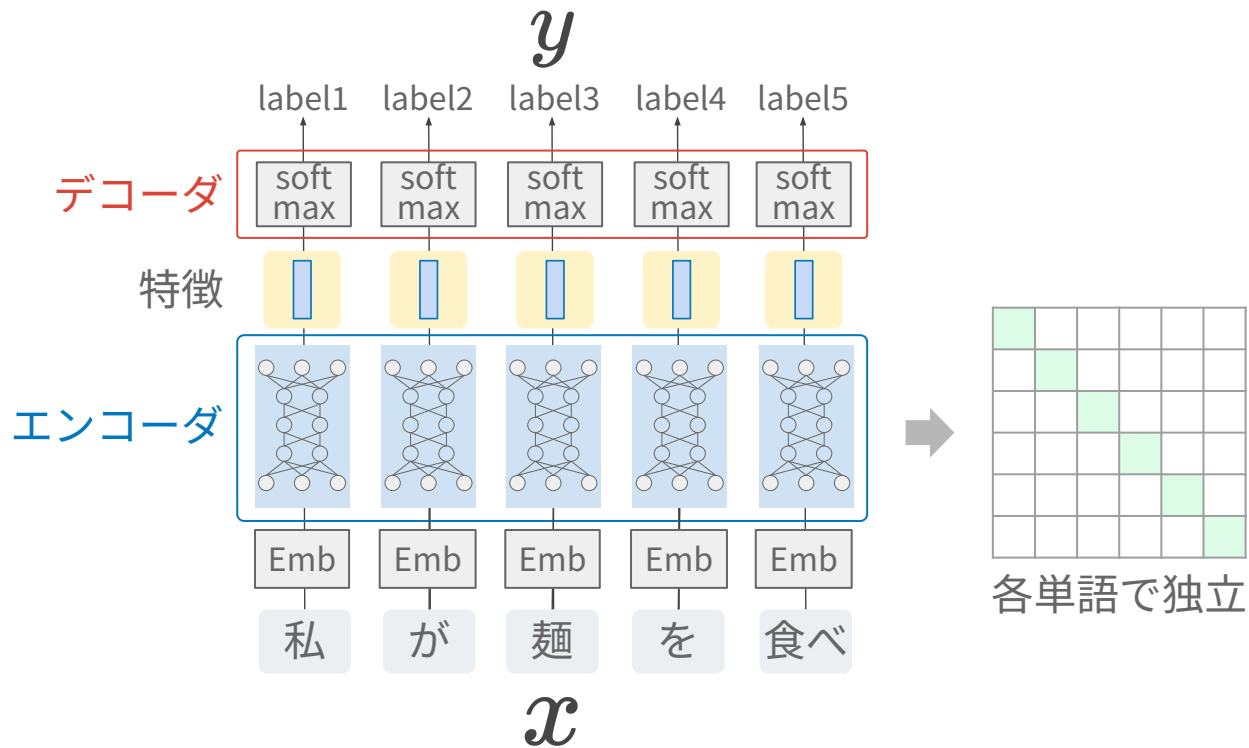


各特徴をどの単語から計算する？

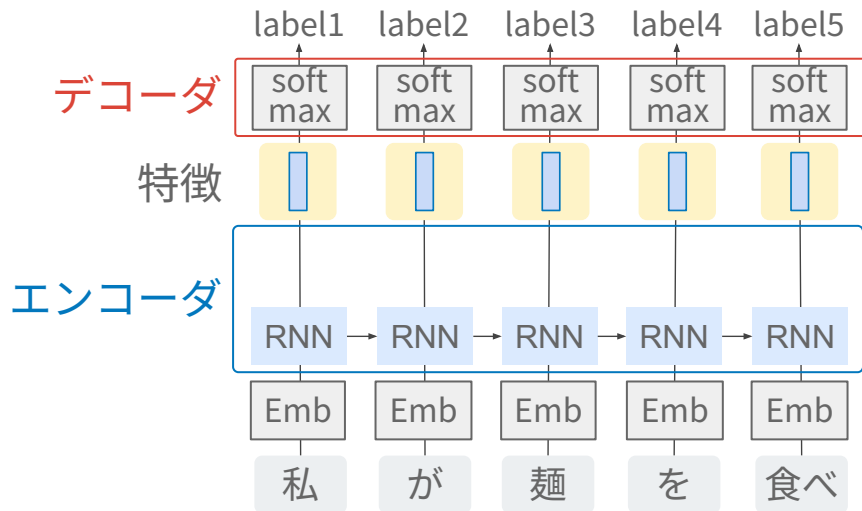
種々のエンコーダとデコーダ

Encoders & decoders

Feed-forward (FF) 型エンコーダ

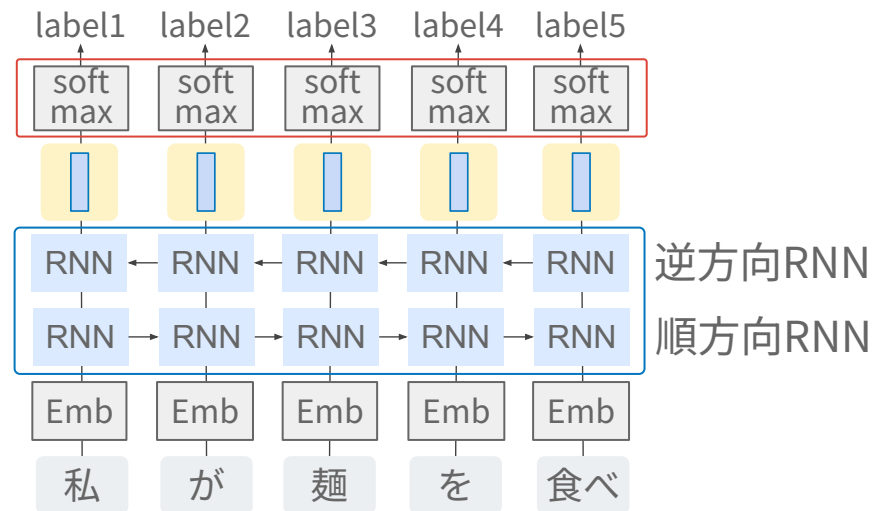
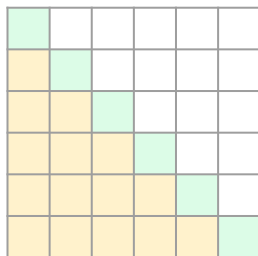


単方向 RNN / 双方向 RNN 型エンコーダ (LSTM, GRU も同様)



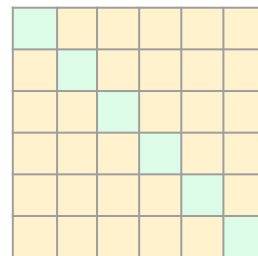
単方向 RNN

過去の全単語から
特徴量を計算

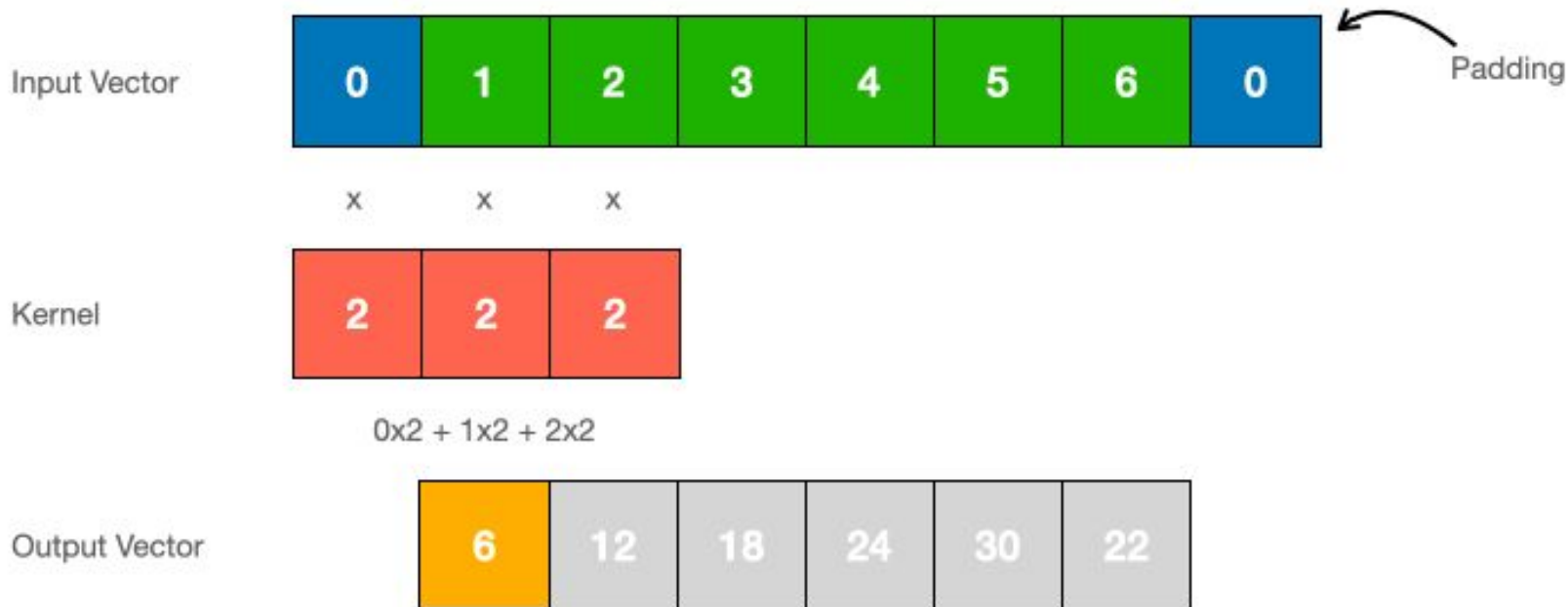


双方向 RNN

文中の全単語から
特徴量を計算

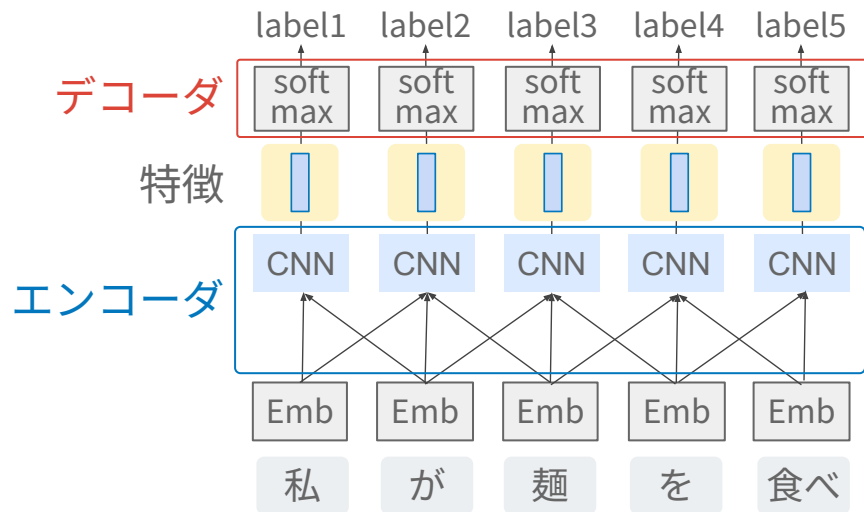


畳み込みニューラルネットワーク (CNN)



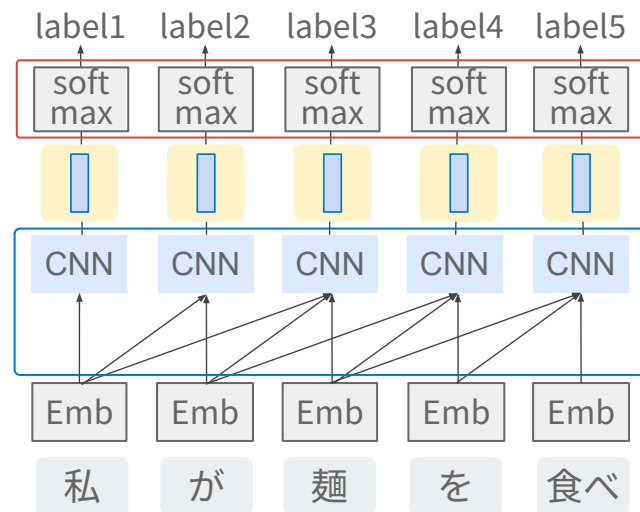
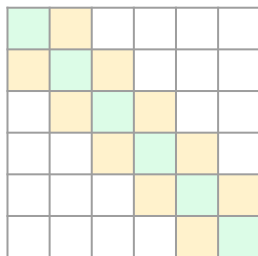
$$y = h \star x$$

非因果性 CNN / 因果性 CNN 型エンコーダ



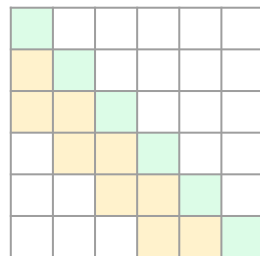
非因果性 CNN

±N 単語から
特徴量を計算



因果性 CNN

過去のN単語から
特徴量を計算



各種エンコーダ + Feed-forward 型デコーダ の式

Feed-forward (FF) 型エンコーダ

$$P(y|x) = \prod_{n=1}^{|x|} P(y_n | \underline{x_n})$$

当該単語のみ

どの単語から特徴抽出する？

- FF は一単語だけから
- RNN は全単語から
- CNN は隣接N単語から

単方向 RNN 型エンコーダ

$$P(y|x) = \prod_{n=1}^{|x|} P(y_n | \underline{x_{<n}}, \underline{x_n})$$

過去的全単語

双方向 RNN 型エンコーダ

$$P(y|x) = \prod_{n=1}^{|x|} P(y_n | \underline{x_1, \dots, x_n}, \underline{x_n, \dots, x_{|x|}})$$

過去的全単語 未来的全単語

非因果性 CNN 型エンコーダ

$$P(y|x) = \prod_{n=1}^{|x|} P(y_n | \underline{x_{n-N}, \dots, x_n}, \underline{x_n, \dots, x_{n+N}})$$

過去のN単語 未来のN単語

因果性 CNN 型エンコーダ

$$P(y|x) = \prod_{n=1}^{|x|} P(y_n | \underline{x_{n-N}, \dots, x_n})$$

過去のN単語

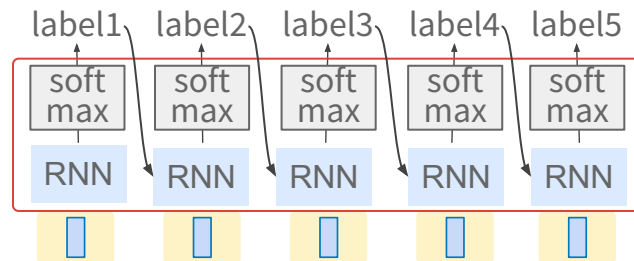
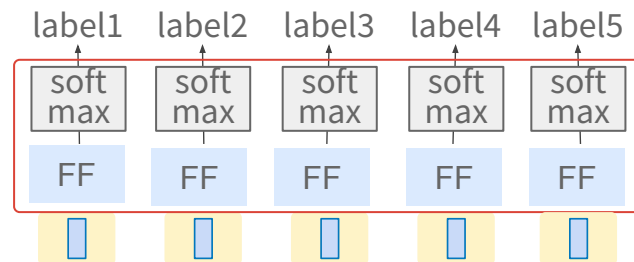
デコーダ側も同様に考えられる

Feed-forward (FF) 型デコーダ

$$P(y|x) = \prod_{n=1}^{|x|} P(y_n|x)$$

RNN 型デコーダ

$$P(y|x) = \prod_{n=1}^{|x|} P(y_n|x, \underbrace{y_{<n}}_{\text{過去の予測}})$$



実際には、RNNの前に Emb 層が入る

最初の系列変換ニューラルネットワーク

Pioneer of sequence-to-sequence conversion with neural networks

Ilya Sutskever et al., “Sequence to Sequence Learning with Neural Networks,” 2014.

系列ラベリングから系列変換へ

系列ラベリング (seq. labeling)

系列全体や各要素につけることを指す

$|y| = 1$
(例：文ラベル)



モデル

私 が 麺 を

$|y| = |x|$
(例：品詞)



モデル

私 が 麺 を

系列変換 (seq.-to-seq. conversion)

可変長の系列へ変換することを指す

一般に $|y| \neq |x|$
(例：翻訳)

I eat ...

モデル

私 が 麺 を

RNN エンコーダ + RNN デコーダ = 系列変換モデル

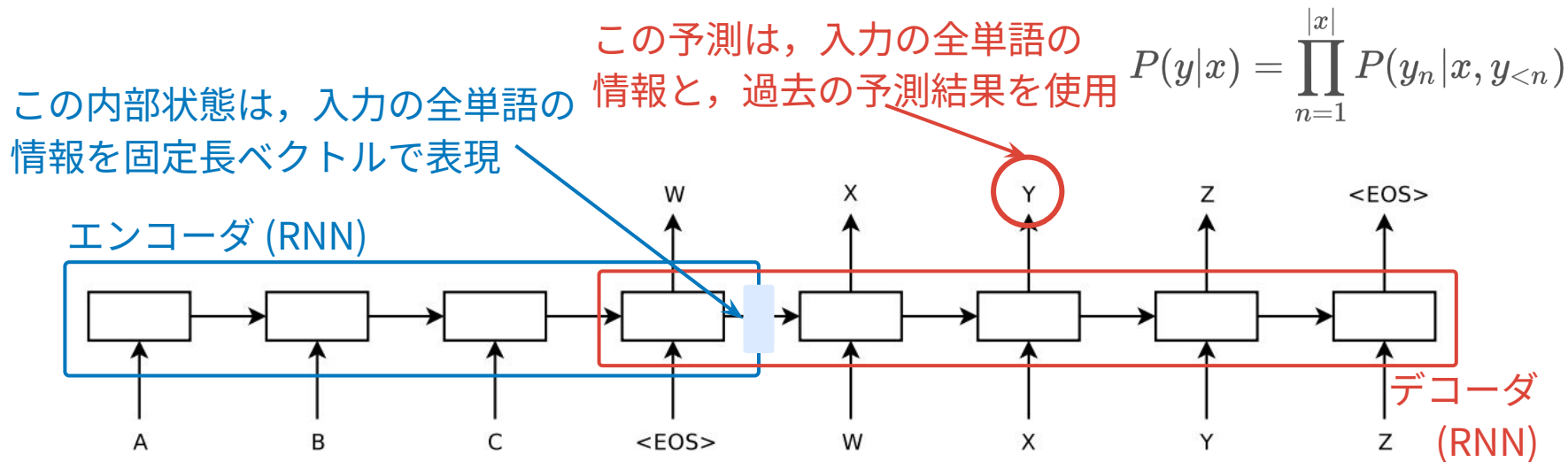


Figure 1: Our model reads an input sentence “ABC” and produces “WXYZ” as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier.

入力と出力の系列長が異なっても変換できる系列変換モデル

実装は RNNLM とほぼ同じ

入力系列と出力系列をまとめて1つの系列として扱えば良い

RNNLM を実践してみよう

<https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/05>

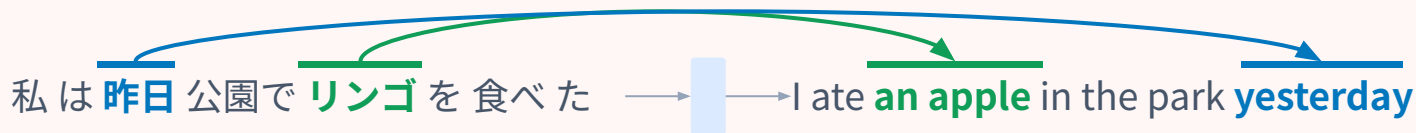
The screenshot shows the GitHub interface for the repository `nlp-lecture-keio`. The left sidebar displays the file tree with the path `2026 / codes / 05` selected. The main content area shows the commit history for this directory, with three commits listed:

Name	Last commit message	Last commit date
..		
data-rnnlm.txt	add rnnlm	11 minutes ago
data.txt	add 05	last week
rnn-lm.ipynb	add .to(device)	1 minute ago

RNNの限界と課題

課題 1：全予測で画一的な特徴を使用

予測のタイミングに応じて、対応する単語の情報だけを取り出せない



全単語の情報は、固定長ベクトルで表現される

課題 2：長期依存性の欠如（情報の忘却）

距離が遠い単語間の関係（主語との対応など）を保持できない

彼が 昨日 ひとりで 図書館 に行って 借りて きた **本**



情報の伝播とともに、初期の情報（「彼」など）が薄れてい

く

注意機構

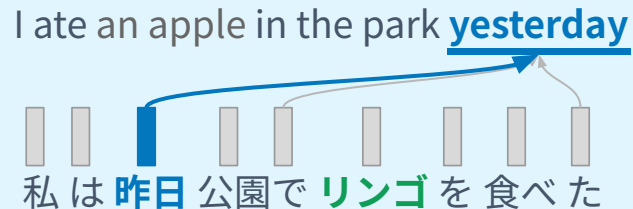
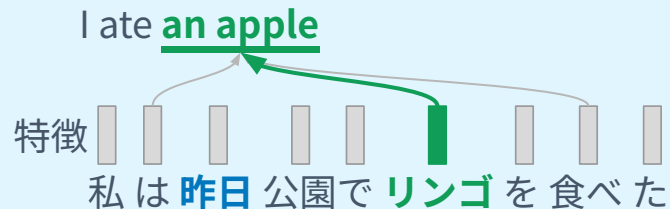
Attention mechanism

Dzmitry Bahdanau et al., “Neural Machine Translation by Jointly Learning to Align and Translate,” 2014. (cross attention, attention を導入した最初の論文)

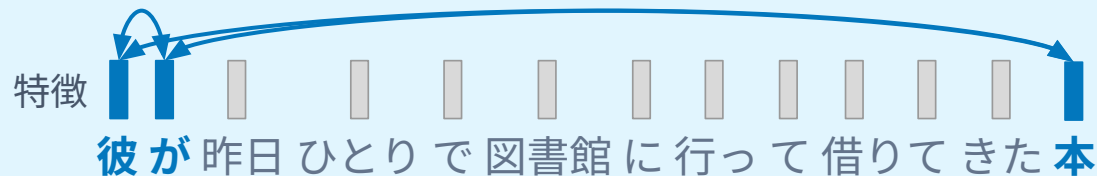
Ashish Vaswani et al., “Attention Is All You Need”, 2017. (self attention, Transformer の論文)

モチベーション

モチベ1：必要に応じて特徴ベクトルを取り出せたらよさそう！



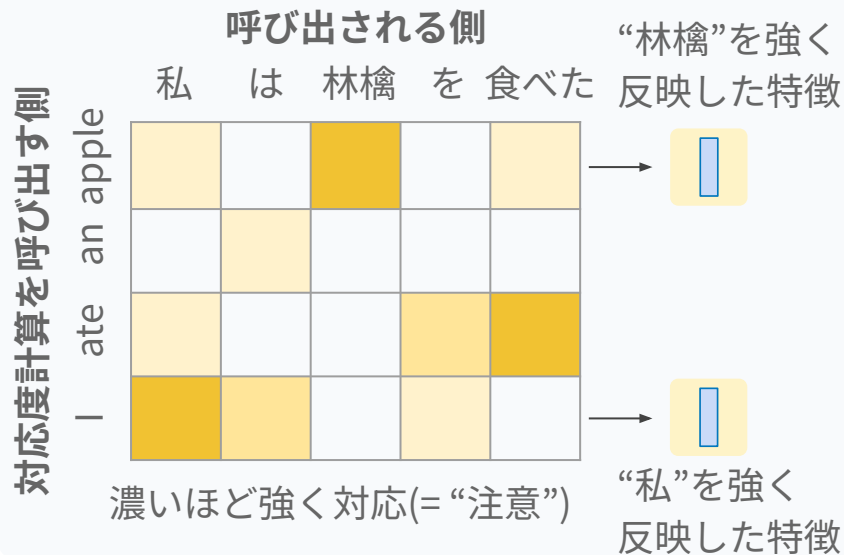
モチベ2：位置に関係なく「関連していること」を特徴づけられたらよさそう！



注意機構の基本的な考え方

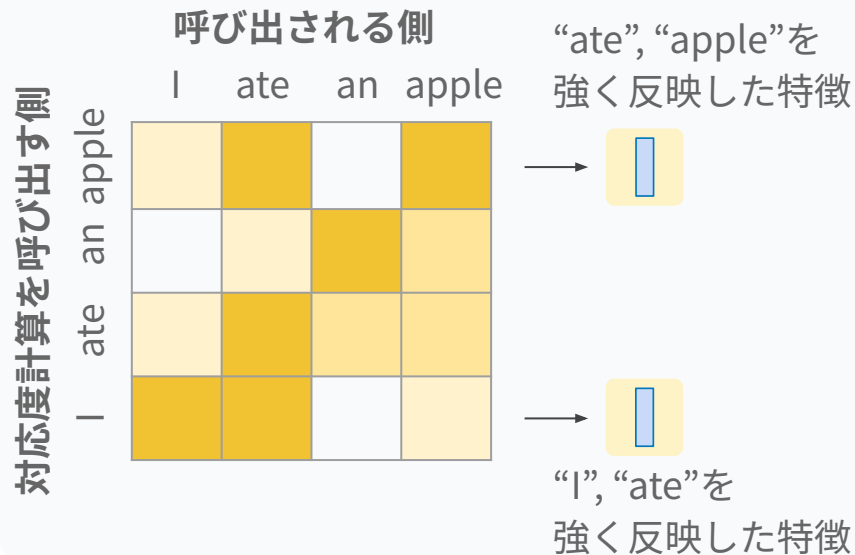
Cross attention

異なる 2 系列 (呼び出す側, 呼び出される側) 同士で関連度を計算．呼び出す側に必要な特徴を随時取り出す．



Self attention

同じ系列 で関連度を計算．特徴同士の関連を計算するため，意味の有る特徴を計算できる．



系列変換における キー と クエリ と バリュー の考え方

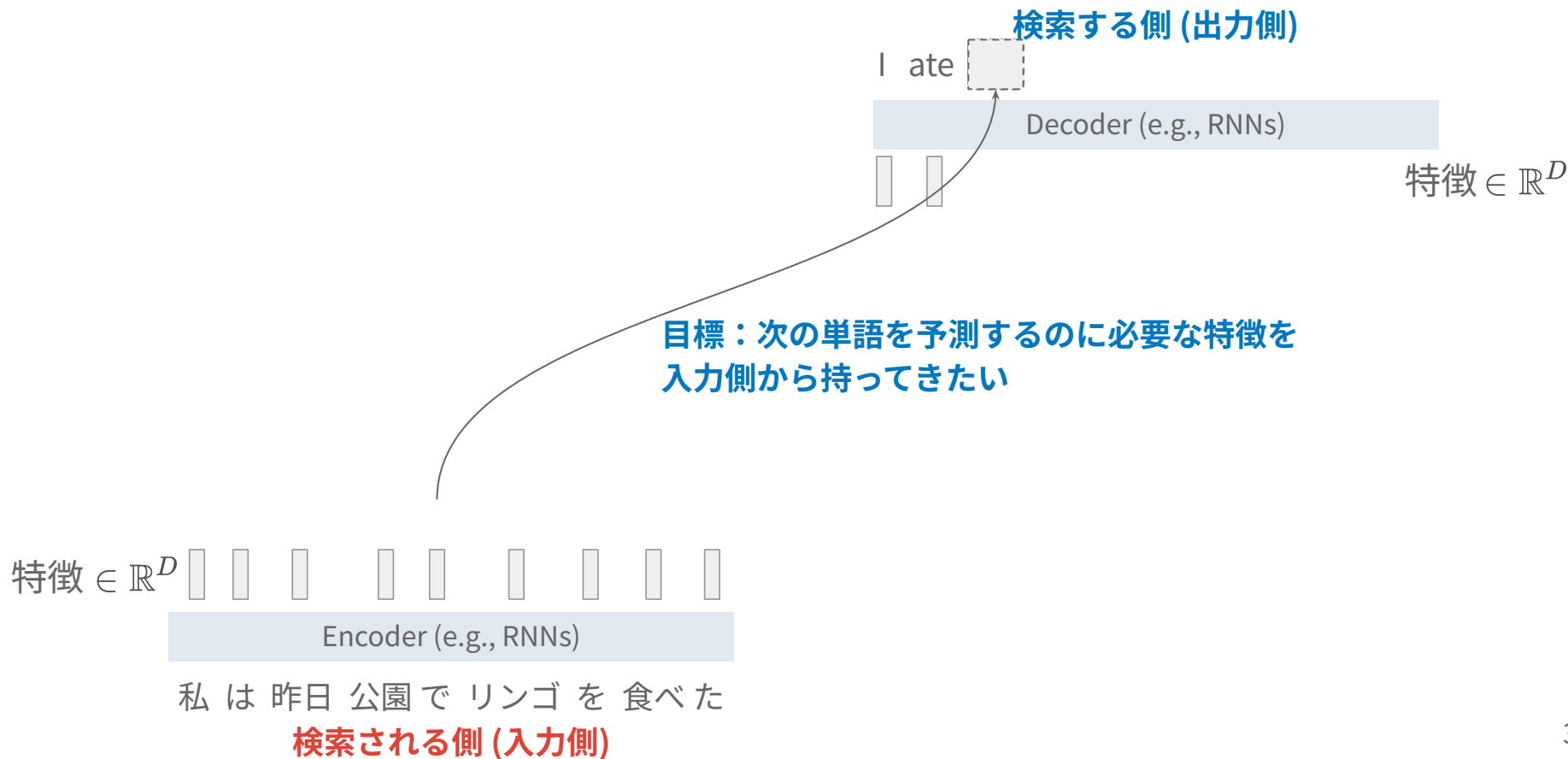
	Cross attention	Self attention
クエリ：検索する側が出す 「検索したいもの」	出力側	入力側
キー：検索される側が出す 「検索対象のヒット (被検索) 要素」	入力側	
バリュー：検索される側が出す 「検索対象の特徴」		

Cross attention を計算してみよう

Let's try the cross attention!

Dzmitry Bahdanau et al., “Neural Machine Translation by Jointly Learning to Align and Translate,” 2014. (cross attention, attention を導入した最初の論文)

Encoder と decoder のそれぞれに各単語の特徴がある



出力側がクエリ，入力側がキーになる

I ate

Decoder (e.g., RNNs)



クエリ

q_2

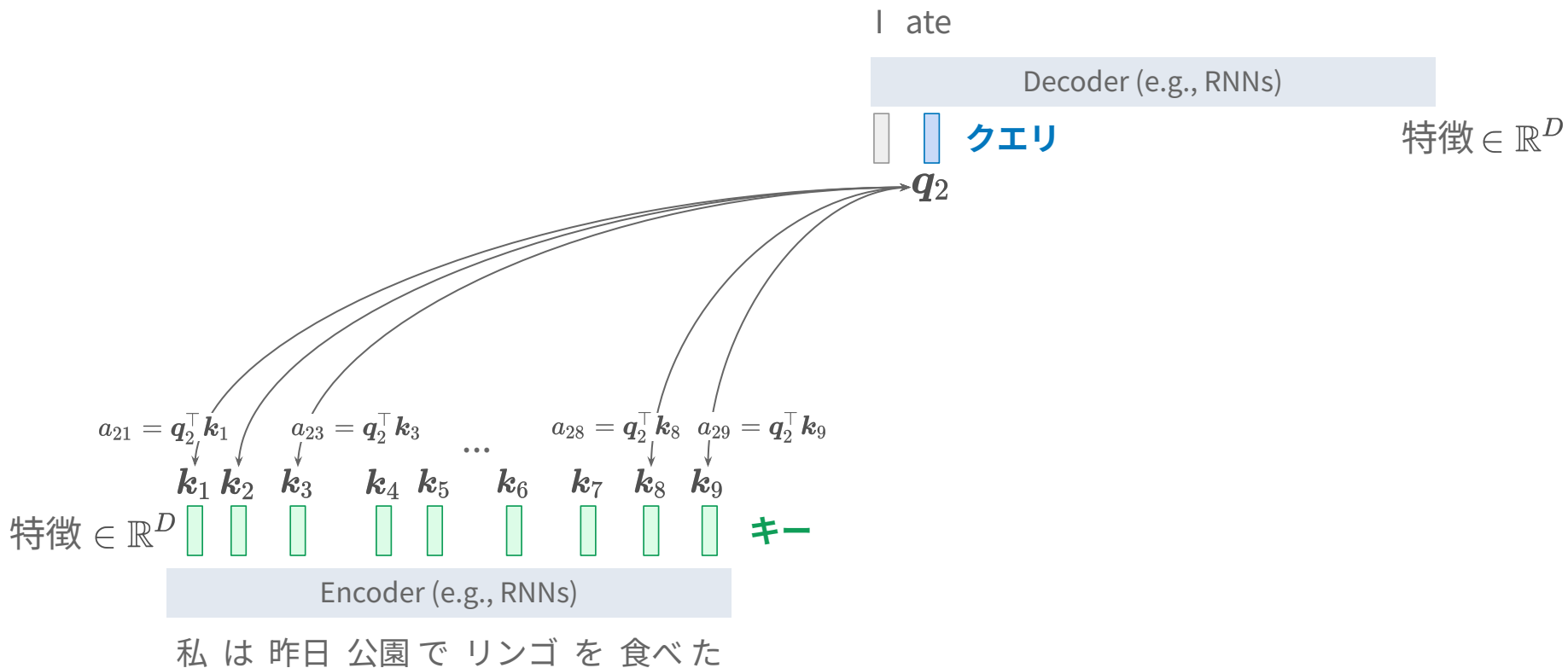
特徴 $\in \mathbb{R}^D$

特徴 $\in \mathbb{R}^D$ k_1 k_2 k_3 k_4 k_5 k_6 k_7 k_8 k_9 キー

Encoder (e.g., RNNs)

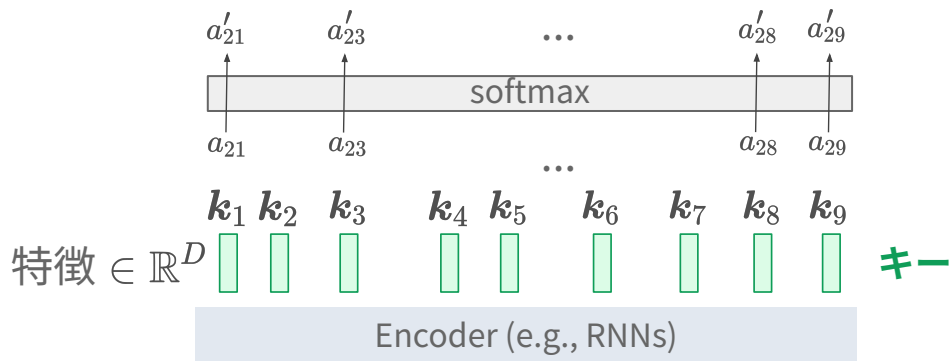
私は昨日公園でリンゴを食べた

クエリとキーの内積をとり対応度を求める



Softmax をかけて正規化 (attention の値にする)

各単語がクエリの単語に
どれくらい関連しているかを表す



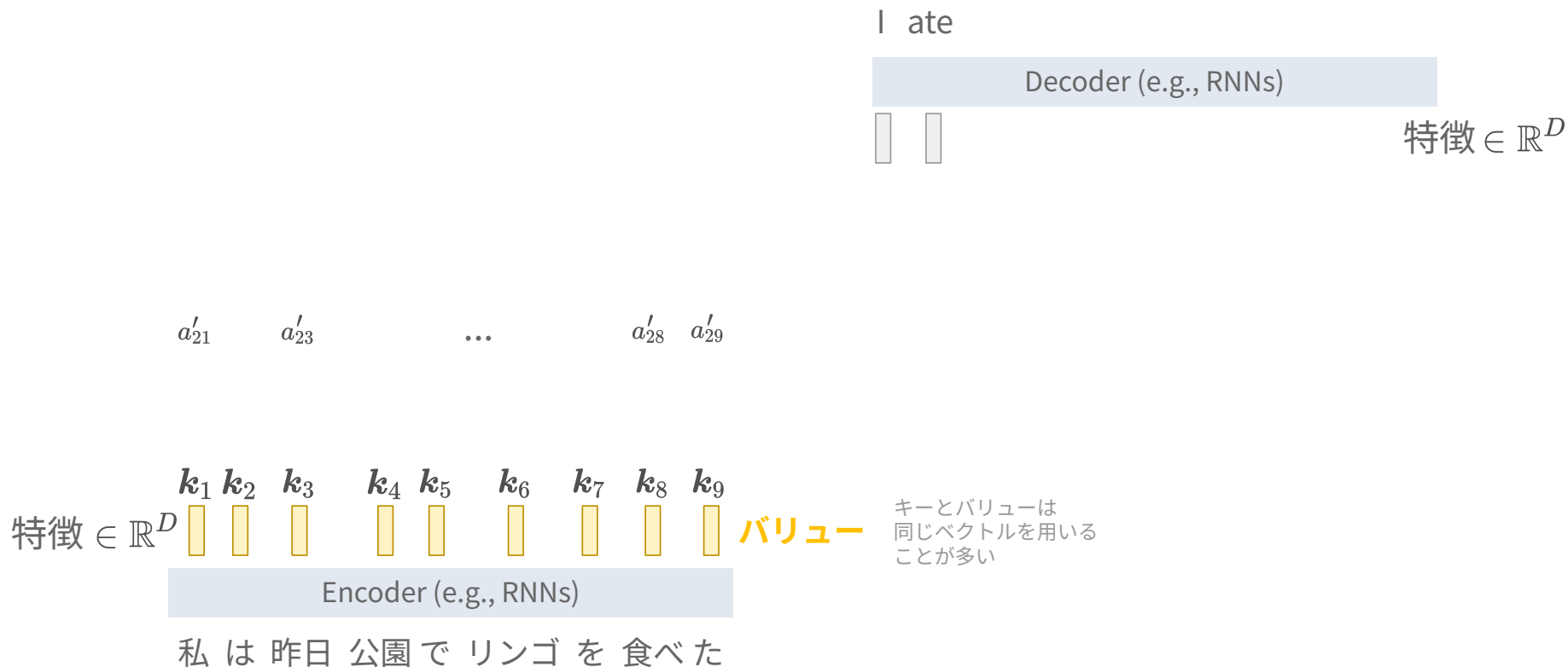
I ate

Decoder (e.g., RNNs)

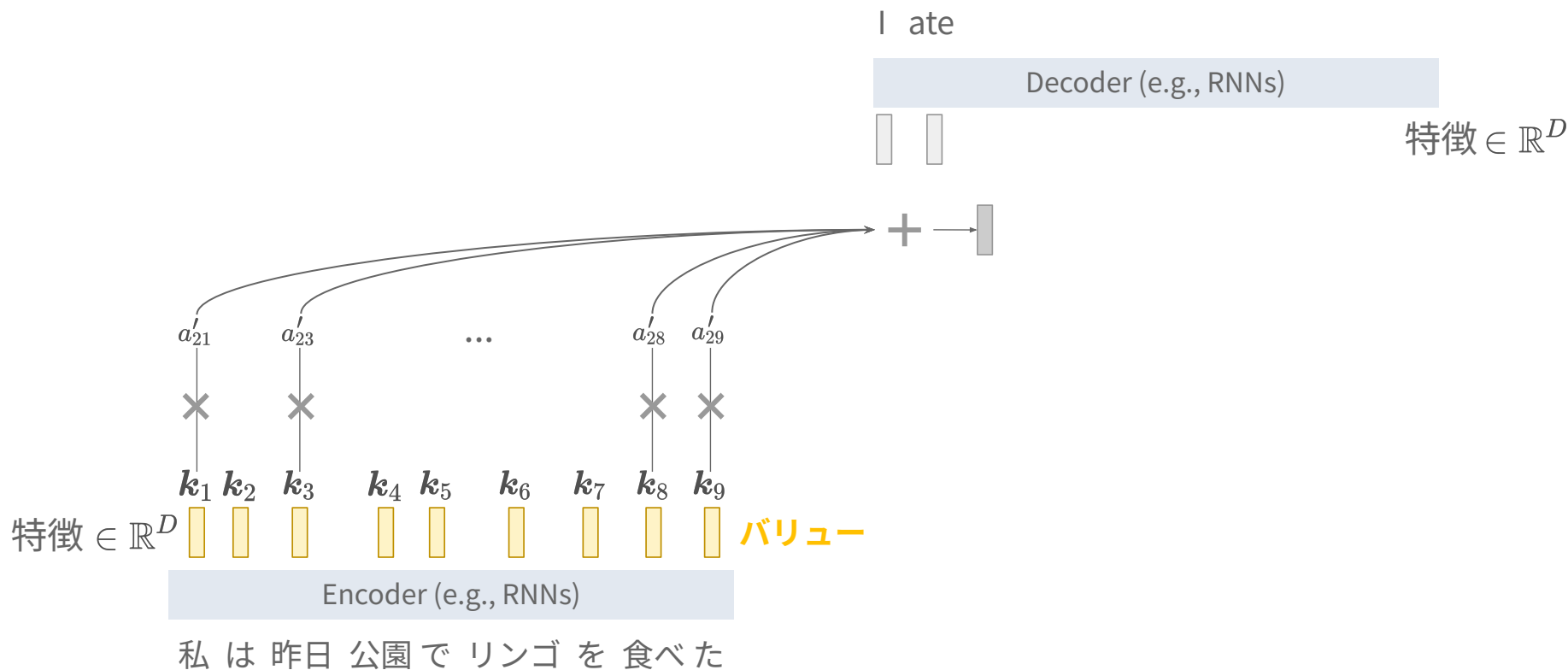


特徴 $\in \mathbb{R}^D$

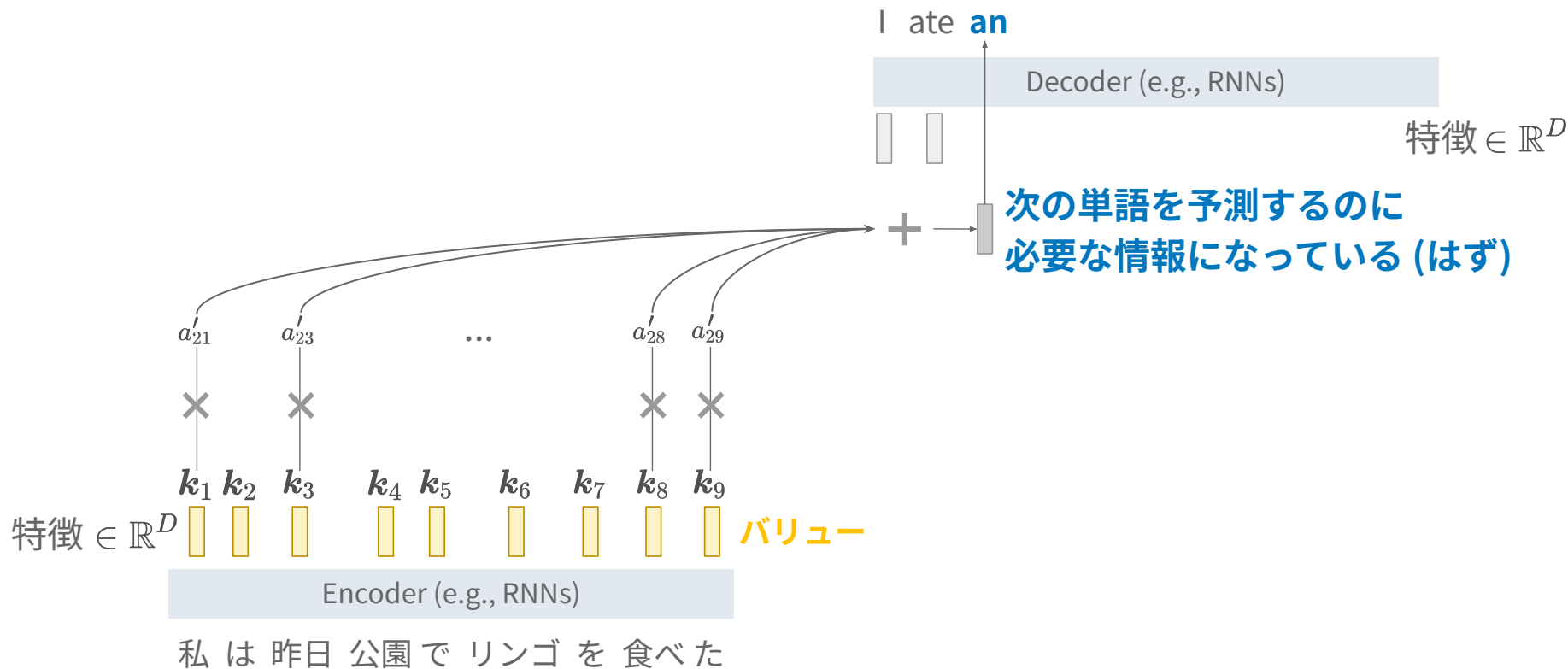
検索される側にバリューを用意する



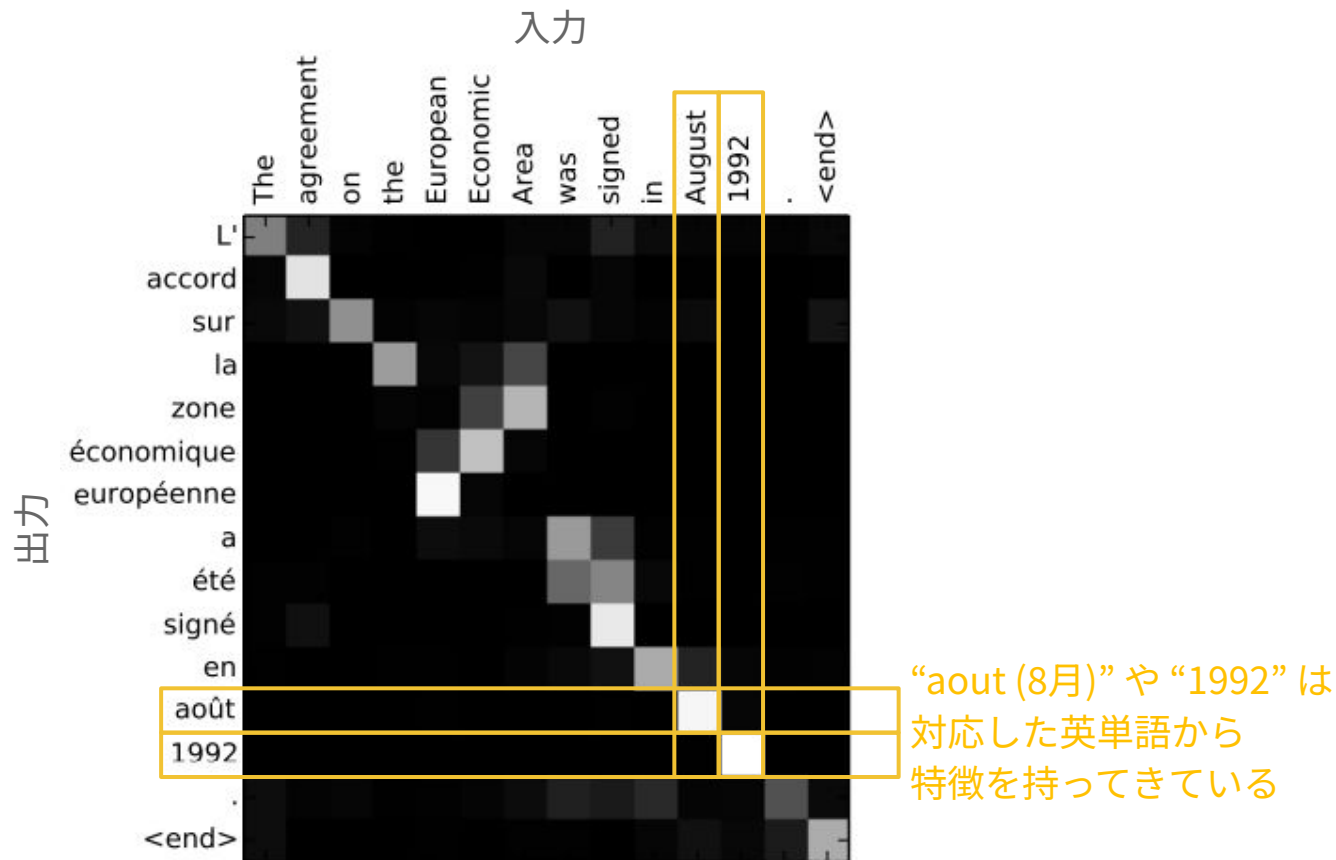
バリューを attention 値で重みづけて足し合わせ



バリューを attention 値で重みづけて足し合わせ



英語→フランス語翻訳における attention 値の行列

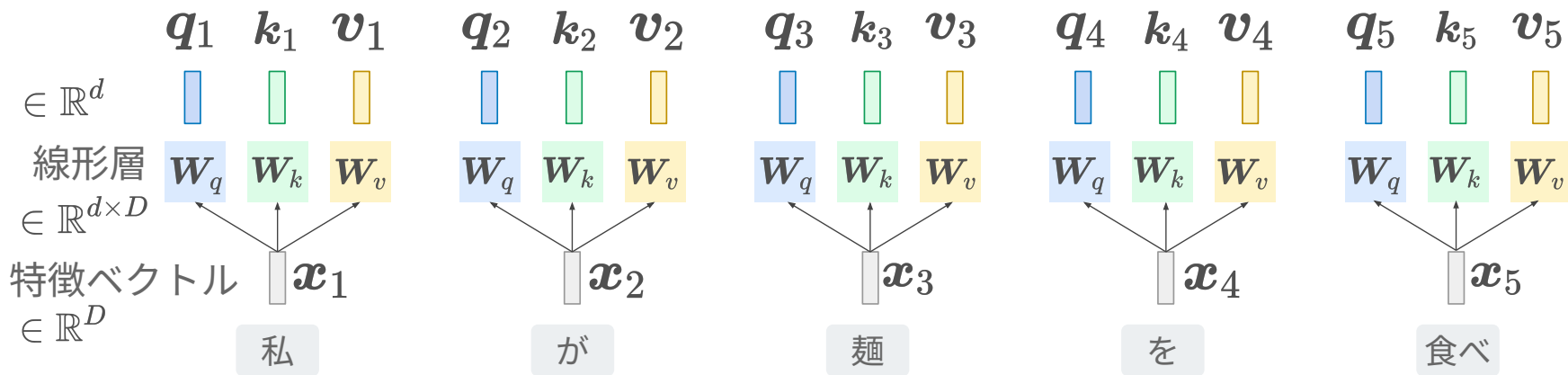


Transformer の self attention (詳細は Transformer の回)

Self attention mechanism in Transformer (details in the next class)

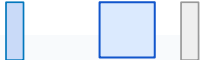
Ashish Vaswani et al., “Attention Is All You Need”, 2017. (self attention, Transformer の論文)

入力の特徴ベクトルからクエリ・キー・バリューを計算



式で書くと

単語ごとに計算する式

$$q_n = W_q x_n, k_n = W_k x_n, v_n = W_v x_n$$


別表記

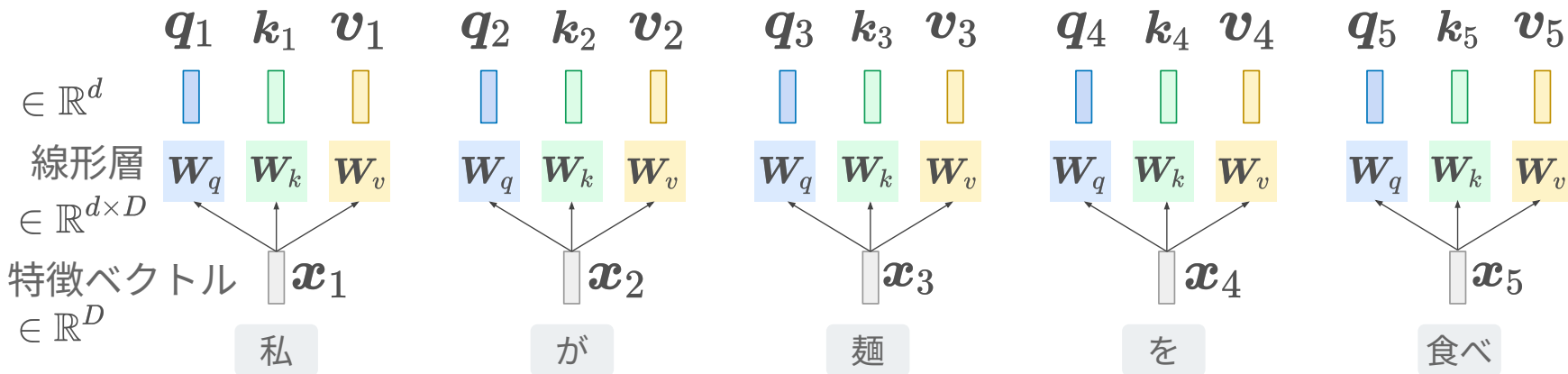


全単語について一括計算する式

$$\begin{bmatrix} q_1^\top \\ \vdots \\ q_{|x|}^\top \end{bmatrix} = \begin{bmatrix} x_1^\top \\ \vdots \\ x_{|x|}^\top \end{bmatrix} W_q^\top$$

$\mathbb{R}^{|x| \times d}$ $\mathbb{R}^{|x| \times D}$ $\mathbb{R}^{D \times d}$

k, v も同様に表記可能



図では $|x|=5$

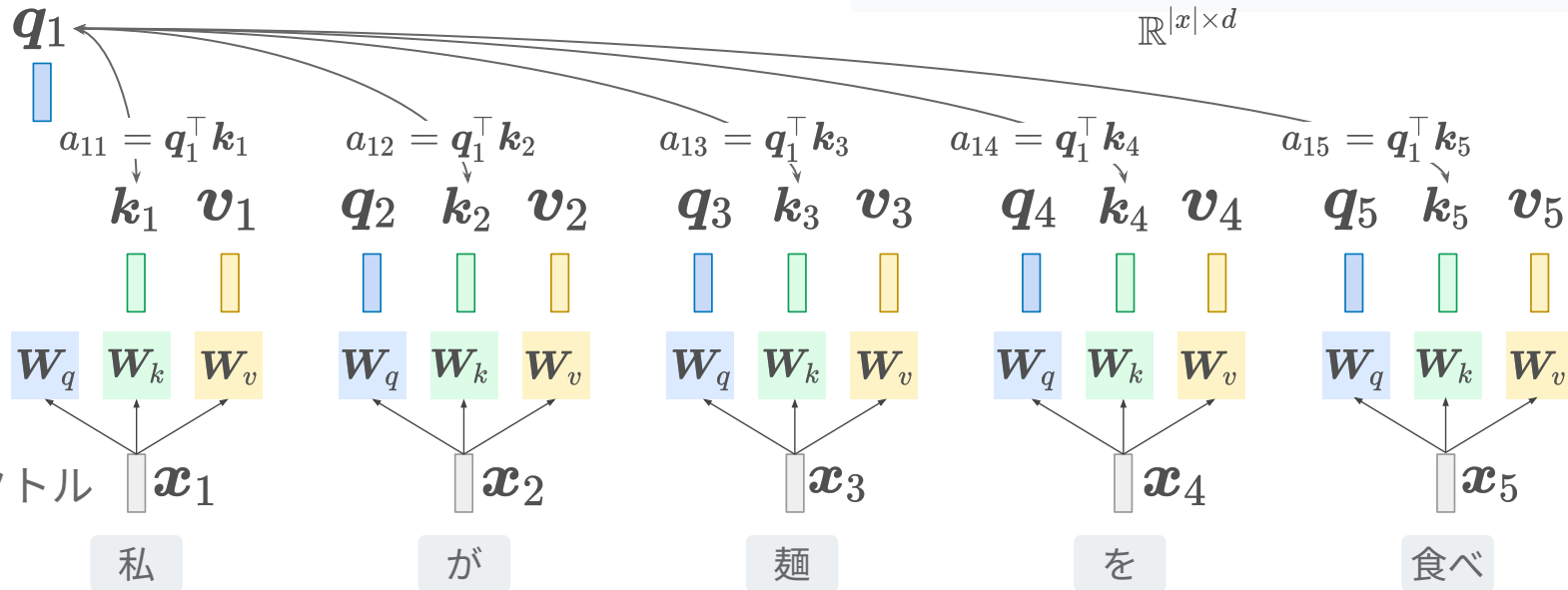
クエリとキーの内積をとり対応度を求める

単語間の内積が入った行列

$$\begin{bmatrix} a_{11} & \cdots & a_{1|x|} \\ \vdots & \vdots & \vdots \\ a_{|x|1} & \cdots & a_{|x||x|} \end{bmatrix}$$

$\mathbf{A} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_{|x|}^\top \end{bmatrix} \begin{bmatrix} \mathbf{k}_1, \cdots, \mathbf{k}_{|x|} \end{bmatrix}$

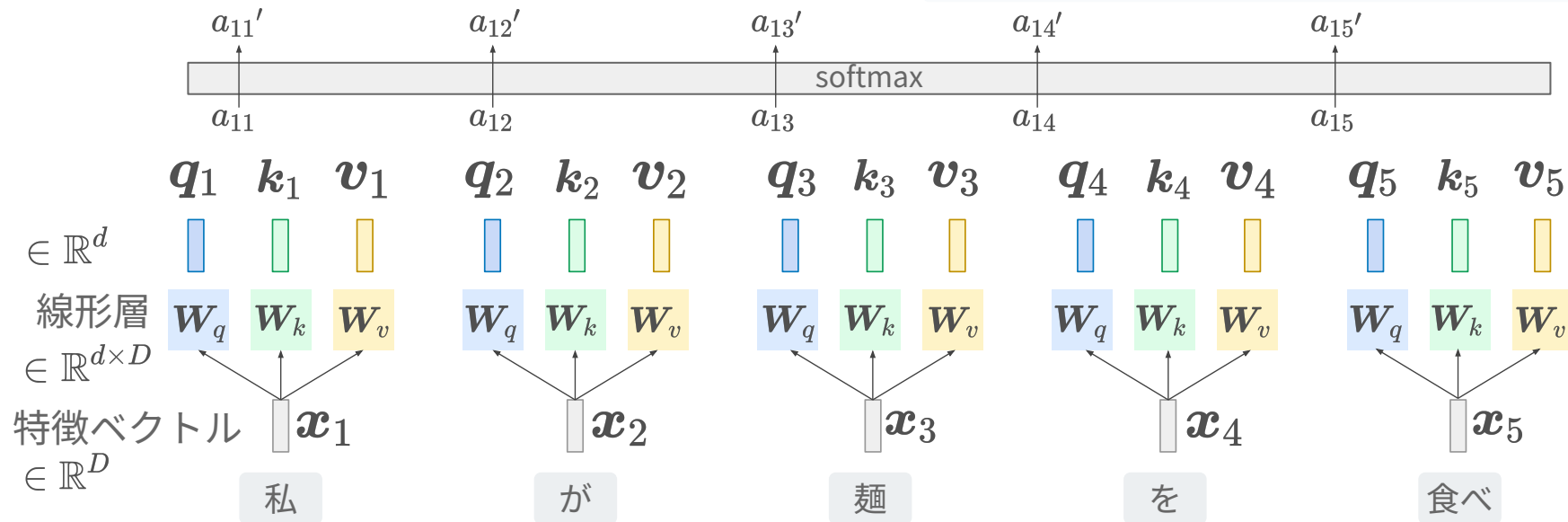
$\mathbb{R}^{|x| \times |x|}$ $\mathbb{R}^{|x| \times d}$ $\mathbb{R}^{d \times |x|}$



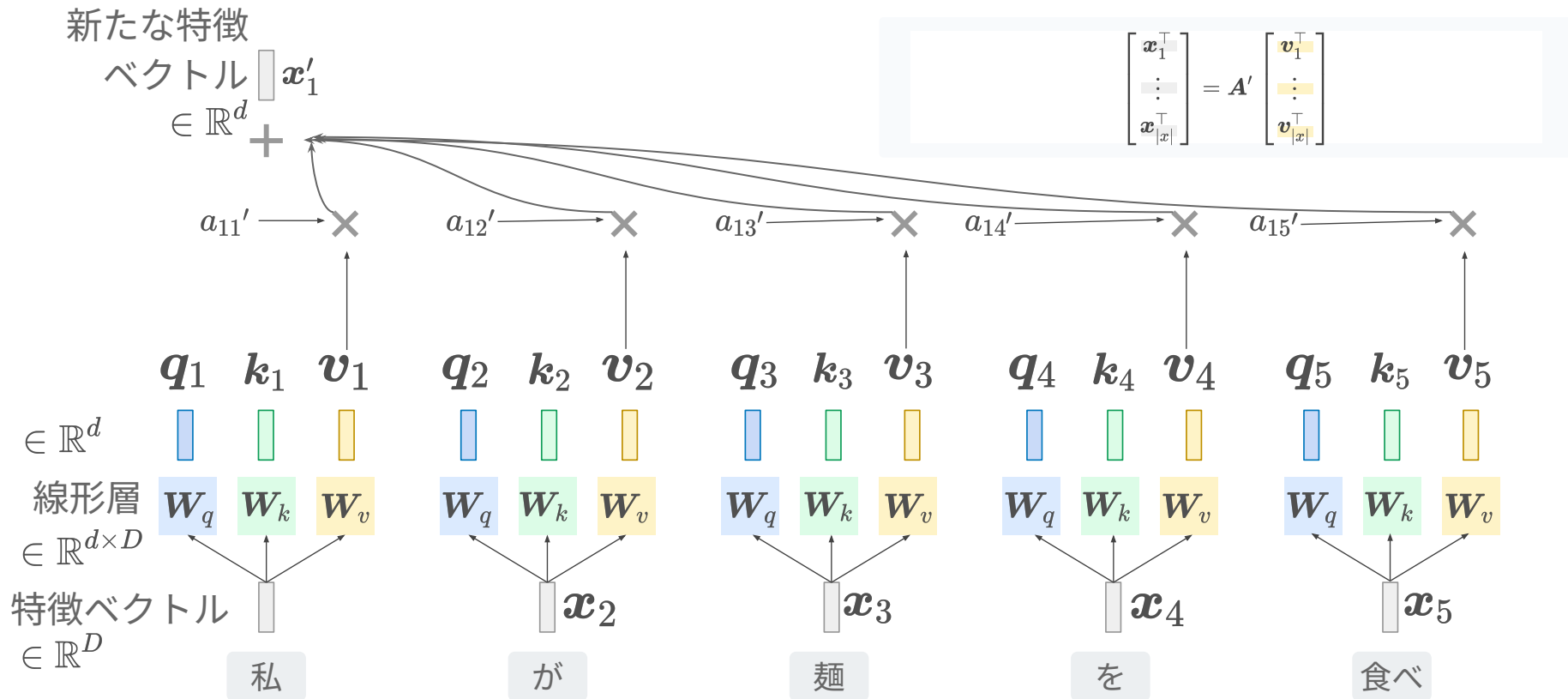
Softmax をかけて正規化 (attention の値にする)

単語間のattention値が入った行列 $\begin{bmatrix} a'_{11} & \cdots & a'_{1|x|} \\ \vdots & \vdots & \vdots \\ a'_{|x|1} & \cdots & a'_{|x||x|} \end{bmatrix}$

$\mathbf{A}' = \text{softmax}_{\text{row}} \mathbf{A}$
 $\mathbb{R}^{|x| \times |x|}$ 行方向のsoftmax (実際には $\mathbf{A}/\sqrt{d}$ に作用) $\mathbb{R}^{|x| \times |x|}$



各バリューに attention 値で重みづけて加算



式でまとめてかくと

入力特徴からクエリ, キー, バリューを線形変換して求める

$$\begin{aligned} \begin{matrix} Q \\ \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{matrix} &= \begin{matrix} X \\ \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times D} \end{matrix} \begin{matrix} W_q^\top \\ \text{blue square} \\ \mathbb{R}^{D \times d} \end{matrix} \\ \begin{matrix} K \\ \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{matrix} &= \begin{matrix} X \\ \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times D} \end{matrix} \begin{matrix} W_k^\top \\ \text{green square} \\ \mathbb{R}^{D \times d} \end{matrix} \\ \begin{matrix} V \\ \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{matrix} &= \begin{matrix} X \\ \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times D} \end{matrix} \begin{matrix} W_v^\top \\ \text{yellow square} \\ \mathbb{R}^{D \times d} \end{matrix} \end{aligned}$$

Attention の値を求めて新たな特徴を得る

$$\begin{matrix} X' \\ \begin{bmatrix} \mathbf{x}_1'^\top \\ \vdots \\ \mathbf{x}_{|x|}'^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{matrix} = \text{softmax}_{\text{row}} \left(\begin{matrix} Q \\ \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{matrix} \begin{matrix} K^\top \\ [\mathbf{k}_1, \dots, \mathbf{k}_{|x|}] / \sqrt{d} \\ \mathbb{R}^{d \times |x|} \end{matrix} \right) \begin{matrix} V \\ \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_{|x|}^\top \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{matrix}$$

QK[⊤]を行ごとに
確率値に

$\sqrt{d}$ の意味は後日

まとめ

まとめ

- 系列ラベリング
 - 系列全体 or 各要素にラベル付けすること
- エンコーダとデコーダ
 - どの入力から特徴を計算するか, どの別出力から出力を求めるか
- 系列変換ニューラルネットワーク
 - RNNエンコーダ + RNNデコーダ
- 注意機構
 - Cross attention & Self attention

課題

1. Gemini や ChatGPT などを利用して、注意機構のコードを生成せよ。
 - a. 既存のコードを用いてもよい。
2. そのコードが注意機構を確かに実装していることを、プログラムへのコメントで説明して提出せよ。
 - a. Jupyter notebook の markdown で書いてもよい。平文でも良い。
 - b. 本講義を受講前の自分自身が理解できるに十分なコメントを残すこと。

```
In [16]: def attention(query, key, value, mask=None, dropout=None):  
         "Compute 'Scaled Dot Product Attention'"  
         d_k = query.size(-1)  
         scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)  
         if mask is not None:  
             scores = scores.masked_fill(mask == 0, -1e9)  
         p_attn = scores.softmax(dim=-1)  
         if dropout is not None:  
             p_attn = dropout(p_attn)  
         return torch.matmul(p_attn, value), p_attn
```

コードの例

第 07 回について

第 07 回について

- 第 01 ~ 06 回の内容を理解しているかを問う課題を出します。
 - Gemini に聞けば答えがわかるように設計しておきます。
- 「情報工学概論」 at 日吉の講義のため、高道は開始 15 分で去ります。
 - 質問があればその時間で聞いてください。
- 回答の提出は不要です。授業後課題也没有ありません。